

Supplementary material: Calibration, proofs and implementation record

Ermal Belul Marwane Bouznif Dritan Nace Antoine Jouglet

This supplement holds the statistical arguments and the secondary results behind the main article; every definition, notation and method statement remains in the main text. Sections S1 to S3 concern the greedy clustering heuristic of Section 4.1 of the main article and the synthetic benchmark of its Section 5: whether the community size bound β scales with catalogue size (N) or with station slot capacity (ζ), how sensitive the solution is to each threshold (ϕ, γ, r, β) , and the paired statistical tests behind the benchmark. Section S4 records the implementation, hardware and parameter settings of every method; Section S5 proves the two bounding claims of the main article; Section S6 records the Company A instance; Section S7 reports the full industrial β search; and Section S8 describes the released per-instance column-generation convergence record.

To isolate these effects without confounding catalogue volume and station capacity (which are tied in the main 29-instance benchmark of Section 5 of the main article), we generated an expanded dataset of 80 synthetic instances across crossed combinations of $N \in \{500, 1000\}$ and $\zeta \in \{20, 25, 50, 100\}$, using the same common-itemset generator script as the primary benchmark but with independent random seeds. This secondary dataset allows us to systematically test whether β scales with N or ζ , and to test threshold robustness across diverse warehouse configurations.

S1 Community bound scaling across warehouse geometries

The three filters of Section 4.1 of the main article are indexed on the catalogue and the community bound on the station. That asymmetry needs evidence, because the benchmark of Section 5 cannot supply it: its generator ties the station count to the catalogue, so $\zeta = 100$ at every size from 500 SKUs upward and the ratio β/ζ is constant across the whole family. Catalogue size and slot capacity are confounded there, and no result measured on it can separate them.

We therefore built a second family in which the two are not confounded. It uses the same generator as Section 5 of the main article (the common-itemset procedure of ? at correlation degree 0.7, with order size drawn uniformly from the integers 5 to 15 and proportional order volumes) and changes one thing: the number of stations is set independently of the catalogue rather than as $\max(5, \lfloor N/100 \rfloor)$. Two catalogue sizes, $N \in \{500, 1000\}$, are crossed with four station counts chosen so that the same slot capacities $\zeta \in \{20, 25, 50, 100\}$ occur at both sizes. Ten instances per cell give 80 in all, generated on seeds 2001 to 2010, disjoint from the seeds of the benchmark. Station labels are drawn from a separate random stream, so at a fixed seed the order structure is identical across the four geometries and only the station configuration changes. Any difference between cells is therefore attributable to geometry alone.

Each instance is solved at every β on the grid $\{2, 3, 5, 8, 10, 12, 15, 18, 20, 25, 30, 40, 50, 75, 100\}$ that falls at or below its own ζ , which gives 980 runs. All 980 returned a feasible layout. Table S1 reports the minimising bound in each cell. Both instance families, with the generator settings and seeds needed to reconstruct them exactly, accompany the article.

Table S1: Visit-minimising community bound β^* across warehouse geometries, on 80 instances disjoint from the benchmark. The same ζ appears at both catalogue sizes.

ζ	N	$ S $	β^*	β^*/ζ	Δ visits vs. $\beta = 15$
20	500	25	12	0.60	−0.72%
20	1,000	50	12	0.60	−0.45%
25	500	20	15	0.60	0.00%
25	1,000	40	15	0.60	0.00%
50	500	10	30	0.60	−0.87%
50	1,000	20	30	0.60	−0.84%*
100	500	5	40	0.40	−1.01%*
100	1,000	10	40	0.40	−1.06%*

β^* minimises mean visits in its cell over the swept values, which include 15. The last column is the change from 15 to β^* , so a negative entry means β^* uses fewer visits and a zero entry means 15 is itself the minimiser, as it is in both $\zeta = 25$ cells. *Separates from 15 at $p < 0.05$ under a Holm-corrected Wilcoxon signed-rank test over the ten instances of the cell. The unstarred non-zero cells fail that test on within-cell variance rather than on effect size: at $\zeta = 50$, $N = 500$ the point effect of −0.87% exceeds that of the significant cell beneath it.

At a fixed slot capacity β^* is the same at both catalogue sizes in all four cells, whereas at a fixed catalogue size it moves with ζ across the whole range. Over the eight cells the Spearman rank correlation between β^* and ζ is 1.000: the two orderings agree exactly. The Pearson correlation on the raw values is 0.962, the shortfall measuring only that the dependence is not linear, as the next paragraph shows. Against N both measures return 0.000. The bound therefore scales with the slot capacity of a station and is insensitive to the length of the catalogue.

The dependence is monotone but not proportional. The minimising ratio β^*/ζ is 0.60 at $\zeta \leq 50$ and falls to 0.40 at $\zeta = 100$, so a single coefficient cannot be optimal everywhere. We adopt 0.4 for two reasons. It is the minimising value at $\zeta = 100$, the capacity of every benchmark instance of 500 SKUs and above and of the sizes at which the method is intended to be used. Where it is not the minimiser it errs low, which matters because larger communities are harder to place within the workload constraint, as Section 6 of the main article shows on a site whose stations are not interchangeable. The price of that choice is the excess of the layout built at $\beta = \lfloor 0.4\zeta + \frac{1}{2} \rfloor$ over the layout built at the cell’s own β^* , paired instance by instance and averaged over the ten instances of the cell. It is nil at $\zeta = 100$, where 0.4ζ is itself the minimiser, and runs from 0.15% at $\zeta = 50$, $N = 500$ to 1.03% at $\zeta = 25$, $N = 500$ across the six cells below it. That quantity is not the last column of Table S1, which measures β^* against the fixed value 15 rather than against the rule.

S2 Threshold sensitivity of the clustering heuristic

Table S2 rescales each of the four thresholds of Section 4.1 of the main article from half to twice its nominal value, one at a time and then all four together, on the twelve 50-SKU and ten 500-SKU instances. The 1,000- and 2,000-SKU instances are omitted because four and three instances are too few for an informative interval, although the ordering of the effects there agrees with what follows. A multiplier applies to the effective nominal value on the instance, that is, after the threshold’s own clamp has been taken (the pre-clamp value comes from N for the three filters and from ζ for the community bound), and the scaled result is clamped again, at 1 for the two frequency floors and at 2 for the community bound. At 50 SKUs both frequency floors already sit on their clamp of 2, so $\times 0.75$ leaves them there and $\times 0.5$ only lowers them to 1; the community bound sits on its clamp of 5, put there by the floor rather than by 0.4ζ , and its downward settings do move it, to 4 and to 2. Entries are paired mean changes against the nominal setting on the same instance. The procedure is deterministic: repeating the nominal setting under five hash seeds returns an identical layout on every instance, so any difference is a real threshold effect and not run-to-run variation. Reporting follows the conventions for

computational experiments with heuristics (??).

The two frequency floors are inert, as rescaling either leaves the visit count unchanged and the surviving edge set bit-identical at every size, because neither removes a product or pair that the kept-edge ratio does not already remove. The ratio changes the edge set substantially but the objective very little, separating from the nominal setting only at $\times 2$. The community bound is the one threshold whose response is not monotone. Halving it costs 0.63% at 500 SKUs and doubling it costs 1.19%, while $\times 0.75$ and $\times 1.5$ are indistinguishable from the nominal setting, while at 1,000 and 2,000 SKUs the pattern is the same, with doubling costing 1.84% and 2.27%. The exception is $\times 1.5$ at 500 SKUs, which recovers 0.45%, consistent with Section S1 above, where the minimising ratio at $\zeta = 100$ is 0.40 but rises to 0.60 at smaller capacities, so a bound half again as large is not yet past the minimum on every instance. A default whose response rises on both sides at the two larger sizes, and is flat rather than falling at the smaller one, is indexed on the right quantity, whereas a default with a monotone response is not. Leaving all four at their defaults is therefore supported here, not assumed.

Table S2: Sensitivity of the clustering heuristic to its four thresholds, on the 22 instances of 50 and 500 SKUs.

Threshold setting	Δ visits, 50 SKUs	Δ visits, 500 SKUs
Frequency floor ϕ , $\times 0.5$ to $\times 2^a$	0.00%	0.00%
Co-occurrence floor γ , $\times 0.5$ to $\times 2^a$	0.00%	0.00%
Kept-edge ratio r , $\times 0.5$	-0.77% [-2.13, +0.58]	+0.16% [-0.09, +0.41]
Kept-edge ratio r , $\times 0.75$	-0.66% [-1.98, +0.65]	0.00%
Kept-edge ratio r , $\times 1.5$	+0.23% [-0.72, +1.17]	+0.08% [-0.27, +0.43]
Kept-edge ratio r , $\times 2$	-0.74% [-2.26, +0.78]	+1.08% [+0.18, +1.98]
Community bound β , $\times 0.5$	+1.77% [-0.17, +3.72]	+0.63% [-0.07, +1.32]
Community bound β , $\times 0.75$	+0.25% [-1.19, +1.69]	+0.03% [-0.46, +0.51]
Community bound β , $\times 1.5$	-0.25% [-2.29, +1.79]	-0.45% [-1.20, +0.30]
Community bound β , $\times 2$	-0.25% [-1.60, +1.10]	+1.19% [+0.25, +2.13]
All four, $\times 0.5$	+1.15% [-1.14, +3.43]	+0.59% [-0.08, +1.27]
All four, $\times 2$	-1.10% [-2.57, +0.38]	+1.78% [+1.02, +2.54]

Entries are mean paired changes in station visits against the nominal setting, with 95% confidence intervals; a positive value means more visits. Every setting returned a feasible layout on every instance. ^aAll four settings give bit-identical output, so the rows are collapsed into one.

S3 Paired statistical testing of the synthetic benchmark

Section 5 of the main article compares the methods on mean station visits, gaps, running time and workload feasibility across the 29 synthetic instances. This section reports the paired statistical testing behind those comparisons, which is kept out of the main text because its resolution is bounded by the instance counts and adds little to the readings the main tables already support.

Instances differ by two orders of magnitude in difficulty, so comparing two methods on their averages would confound the between-method difference with the between-instance variance. We therefore compare them instance by instance. For each pair of methods we take the per-instance difference in station visits and test whether those differences are systematically signed. The test is the two-sided exact Wilcoxon signed-rank test (?), the standard choice for comparing algorithms across a set of benchmark instances because it imposes no distributional assumption on the differences (?), and ties are discarded. The reported p is the probability, under the null hypothesis that the paired differences are symmetric about zero, of observing a signed-rank statistic at least as extreme as the one obtained, in either direction. Table S3 reports the outcome against the set-variable formulation, the per-instance reference of the main article.

Table S3: Paired Wilcoxon signed-rank tests on the 29 synthetic instances, against the set-variable formulation.

Comparison	Size (SKUs)	n	Wins	p	Min. p
Binary MILP vs. reference	50	12	4	0.74	0.0005
	500	10	8	0.049	0.0020
	1,000	4	1	0.38	0.125
	2,000	3	0	0.25	0.25
	pooled	29	13	0.91	–
CG vs. reference	50	12	1	0.0010	0.0005
	500	10	2	0.0273	0.0020
	1,000	4	2	1.00	0.125
	2,000	3	3	0.25	0.25
	pooled	29	8	0.18	–
GA and SA-C vs. reference	50	12	0	0.0005	0.0005
	500	10	0	0.0020	0.0020
	1,000	4	0	0.125	0.125
	2,000	3	0	0.25	0.25
Heuristic vs. reference	50	12	0	0.0005	0.0005
	500	10	0	0.0020	0.0020
	1,000	4	0	0.125	0.125
	2,000	3	3	0.25	0.25

“Wins” counts instances on which the first-named method returns fewer visits. “Min. p ” is the smallest two-sided p attainable at that sample size, 2^{1-n} , and where p equals it the test is saturated rather than failed, so the win count carries the evidence. The GA, SA-C and Heuristic rows coincide at 50, 500 and 1,000 SKUs because none of the three wins a single paired instance there. The Heuristic separates from them only at 2,000 SKUs, where it wins all three.

No test at the two larger sizes can resolve anything. The resolution of a signed-rank test is bounded by the number of instances available, and on this benchmark that bound is binding: with four and three paired instances the smallest attainable two-sided p is 0.125 and 0.25, so at 1,000 and 2,000 SKUs no result, however unanimous, reaches the conventional 0.05 threshold. Not every cell there is even saturated: the two rows at $p = 0.38$ and $p = 1.00$, both at 1,000 SKUs, sit above their floor of 0.125 because the instances are split rather than unanimous, whereas a cell that does attain its floor is as decisive as its sample permits. At both sizes we therefore treat the win count as the primary evidence, which is the form in which the main article states them.

The separations are reported without correction for multiplicity. Of the seven separations below 0.05 that the table shows, the five at $p \leq 0.0020$ survive a Bonferroni, Holm or Benjamini–Hochberg correction over the eighteen tests reported, whereas the two at $p = 0.0273$ and $p = 0.049$ do not. A small p indicates that the methods differ, whereas a large p indicates only that these instances are insufficient to separate them, which is a weaker statement than equivalence. Effect size and consistency are therefore reported separately: the Gap column of the main article’s benchmark table quantifies the magnitude of a difference, and the test quantifies how reliably it recurs.

S4 Implementation, hardware and baseline parameters

This section states in full the computational setup that Section 5.2 of the main article summarises, together with the incremental swap evaluation of Section 4.2.3, the stage budgets of the price-and-complete drive, and the parameters of the two literature baselines.

S4.1 Operators of the price-and-complete drive

Section 4.2.3 of the main article describes the four operators of its Algorithm 1 in words. This subsection gives the rule each one applies, with $\Pi(s)$ the products a layout Π places at station s , σ_p and μ the duals of the covering and cardinality rows, and B the time cap.

DESCEND minimises the penalised visit count

$$\Psi(\Pi) = \sum_{u \in U} w_u |\{s \in S : \Pi(s) \cap u \neq \emptyset\}| + \rho \sum_{s \in S} \max(0, \text{load}(s) - T_s), \quad \rho = \sum_{u \in U} w_u,$$

the visits counted over the distinct multi-item supports plus ρ times the total workload excess, so that no exchange can lower Ψ by trading feasibility for visits. It exchanges two products held at different stations, which leaves every station's product count unchanged, and accepts the first exchange that lowers Ψ . Each sweep visits the catalogue in a fresh random order; a product is paired with every later product when the catalogue holds at most 300, and with 48 partners drawn at random beyond that. The sweep restarts from the improved layout after each accepted exchange, and the descent stops when a full sweep improves nothing or its budget expires, $\min(15\text{ s}, 0.08B)$ on the warm start and $\min(2\text{ s}, 0.02B)$ on each completion inside the generation loop. Section S4.2 gives the incremental evaluation that makes one exchange cheap to score.

GREEDYPRICE returns one bundle per master solve. Its candidates are the products of strictly positive dual price, ordered in two ways, by decreasing σ_p and by decreasing σ_p per unit of workload L_p/V . Along each ordering it extends a bundle while the slot and line limits hold, tracking the exact reduced cost after every addition, and keeps the prefix of least reduced cost; the better of the two prefixes is returned, and nothing is returned when neither is negative.

EXACTPRICE solves the pricing subproblem of Section 4.2.2 of the main article as an integer programme under a per-call time limit. It returns the bundles found, the solver's dual bound rc_{lb} , which remains valid when the call is truncated, and a flag recording whether the call reached proven optimality.

COMPLETE extends a priced bundle to a full layout. It takes the remaining products in decreasing L_p and gives each to the station that admits it under both caps and whose products already touch the greatest weight of supports containing it, ties going to the lowest-indexed station. A product no station admits under the workload cap goes to the least loaded station, which can leave the completion infeasible; the gate of Stage 2 then discards it.

The warm start of Stage 1 seeds the pool with the LPT partition and 20 mutated variants of it. Each variant displaces a random 5 to 10% of the product-to-station assignments, reinserts the displaced products in random order at the first station of a random permutation that admits them under both caps, and drops any product no station admits, so no infeasible column enters the pool.

The pricing model is built once per instance: its constraint matrix does not depend on the duals, so each call rewrites only the objective coefficients of the a_p variables and, on the station-indexed master, the two resource limits. The build costs about 21 seconds on a 2,000-SKU instance; the solves it serves dominate, and the number of pricing iterations reached inside the cap is what limits the method.

S4.2 Incremental evaluation of the swap descent

The descent of Section 4.2.3 of the main article keeps a support-count matrix $\text{cnt}[i][u]$, the number of products of station i contained in support u . Station i visits u exactly when $\text{cnt}[i][u] > 0$. Removing a product frees the supports whose count falls to zero and inserting it costs those whose count rises from zero. Only supports containing the two moved products can change, so a swap is scored in time proportional to their support degrees rather than by re-scanning the orders, and the counts are updated the same way once a swap is accepted. Beyond about 300 products the candidate partner set is sampled to keep the cost of one descent step in hand.

S4.3 Stage budgets of the price-and-complete drive

The time cap B of Section 4.2.3 of the main article is divided across the four stages of the drive, and Table S4 gives the two schedules used. The heterogeneous industrial run is matched to its longer runs: it allots smaller shares to the warm start and the generation loop than the homogeneous benchmarks do, and it inserts a bound pass that the homogeneous schedule does not use.

Table S4: Stage budgets of the price-and-complete drive, as cumulative fractions of the time cap B .

Stage	Homogeneous benchmarks	Industrial run
Warm start	$0.08B$	$0.05B$
Generation loop	$0.70B$	$0.62B$
Bound pass	not used	$0.74B$
Integer master	$0.85B$	$0.82B$
Final polish	remainder	remainder

Each entry is the point in the cap at which that stage ends, so the generation loop of the homogeneous schedule runs from $0.08B$ to $0.70B$. The final polish takes whatever the integer master leaves.

S4.4 Baseline operators and parameters

Both baselines score a candidate layout with the same surrogate: the station-visit count, plus a penalty on the total excess over slot capacity or workload cap, plus a dispersion term on the co-occurrence weight of product pairs placed at different stations. The first term is the real objective, the second makes an infeasible layout uncompetitive, and the third breaks ties between layouts the visit count cannot separate. Table S5 states the weights of that surrogate and the operator settings of both procedures.

The genetic algorithm uses binary-tournament parent selection, crossover copying a contiguous segment of one parent into the other with a repair pass for over-capacity stations, mutation exchanging two products' stations, and the best layout of each generation carried forward unchanged. Each initial member is drawn by assigning products to stations uniformly at random subject to ζ_s , as ? prescribe.

Simulated annealing with correlated interchange cools geometrically until its stopping temperature is reached. A move is drawn from the correlated pair list, co-occurring pairs being sorted by descending co-occurrence and split into equal groups from which a group and then a pair are drawn uniformly, and the two products exchange stations. A move lowering the surrogate is always accepted, whereas one raising it by Δ is accepted with probability $\exp(-\Delta/T)$. The run starts from a cube-per-order-index (COI) layout, which is correlation-blind like LPT but ranks products by demand volume per unit of storage occupied rather than by order-line frequency, filling stations in that order (?).

Table S5: Surrogate weights and operator settings of the two literature baselines.

Parameter	Value
<i>Shared surrogate</i>	
Excess penalty on slot capacity and workload cap	1,000
Dispersion weight on separated correlated pairs	0.01
<i>Genetic algorithm (?)</i>	
Population size	50
Maximum generations	200
Crossover probability	0.8
Crossover segment cap	$0.25 P $
Mutation probability	0.1
<i>Simulated annealing with correlated interchange (?)</i>	
Initial temperature	800
Geometric cooling ratio per level	0.95
Stopping temperature	1
Moves proposed per level	$\min(2 P , 1000)$
Correlated-pair groups	10

Both procedures also stop at the per-size wall-clock cap of Section 5.2 of the main article, whichever comes first. The crossover segment cap of $0.25 |P|$ is a quarter of the catalogue. Seeds and solver versions are given in Section S4.5.

S4.5 Hardware, solver versions and seeds

Every run reported in the main article was produced on the same remote machine, a 16-core Windows Server 2019 host with 128 GB of memory, under the same Python 3.10 environment, and every method received the same core and thread allocation, four threads, so no method was given more compute than another under the shared wall-clock cap. The column generation runs on CPLEX 22.1.1 and the two MILP representations on Hexaly 13.0. One run per instance is reported in every cell. The two baselines are our own Python implementations of the published operators, so a residual implementation-efficiency difference against the two commercial engines cannot be excluded, and we record how far each baseline got rather than assume it exhausted its search.

Both metaheuristic baselines draw their search decisions from a NumPy generator seeded at 42, so SA-C is fully determined by the instance and the cap. The genetic algorithm’s capacity repair draws instead on the global random stream, seeded at 20240612 by the experiment harness, and that stream is the only source of run-to-run variation in the reported GA figures. The column-generation drive is also stochastic, since Stage 1 mutates the warm start and the descent samples candidate partners beyond about 300 products, and it draws on the same harness stream. We report one run per cell for it as well, so the 2,000-SKU result rests on three single draws and a replicated cell would strengthen it.

S5 Proofs of the bounding claims

S5.1 Binary MILP linear relaxation floor

Take any point feasible for the linear relaxation. For an order $o \in O$ and a product $p \in P_o$, the linking row $x_{ps} \leq z_{os}$ forces $z_{os} \geq x_{ps}$ for every station, and summing over the stations together with the covering row $\sum_{s \in S} x_{ps} \geq 1$ gives $\sum_{s \in S} z_{os} \geq \sum_{s \in S} x_{ps} \geq 1$. The objective therefore cannot fall below $|O|$ at any feasible point, fractional or integral. That floor is attained: setting $x_{ps} = 1/|S|$ meets the capacity rows whenever $|P|/|S| \leq \zeta_s$ and the workload rows under

the operational slack of Section 5.1 of the main article, so $z_{os} = 1/|S|$ is feasible and the objective equals $|O|$ exactly. On this instance family the relaxation is therefore worth precisely the statement that every order visits at least one station, independent of the co-occurrence structure that makes one instance harder than another, so a branch-and-bound search starts from that value with nothing instance-specific to prune with.

S5.2 Column generation Farley-type dual bound

The bound does not rest on the price-and-complete drive. It follows from linear-programming duality applied to the restricted master together with a valid dual bound from the pricing solve, and would hold whatever heuristic produced the layouts.

Section 4.2.1 of the main article aggregates the station-indexed master over interchangeable stations. Written over the current pool Q' of feasible bundles, with c_q the weighted-support cost of bundle q , that restricted master is

$$\begin{aligned} \min \quad & \sum_{q \in Q'} c_q y_q \\ \text{s.t.} \quad & \sum_{q \ni p} y_q \geq 1 & \forall p \in P \quad (\sigma_p \geq 0) & (1) \\ & \sum_{q \in Q'} y_q \leq |S| & (\mu \leq 0) & (2) \\ & 0 \leq y_q \leq 1 & \forall q \in Q'. \end{aligned}$$

The upper bounds $y_q \leq 1$ may be dropped, since reducing any y_q above one to one preserves coverage, relaxes the cardinality row and cannot increase the objective, so an optimal dual solution with zero multipliers on them exists. Strong duality gives $z_{\text{RMP}} = \sum_p \sigma_p + |S|\mu$. The pair (σ, μ) need not be dual-feasible for the full master, because a bundle outside the pool may price out negatively. Let $\text{rc}_{\min}$ be the least reduced cost over all feasible bundles and suppose it is negative. Replacing μ by $\mu' = \mu + \text{rc}_{\min}$ restores feasibility, since every bundle then satisfies $c_q - \sigma(q) - \mu' = \text{rc}_q - \text{rc}_{\min} \geq 0$, and μ' remains non-positive. Weak duality at this repaired point gives

$$z_{\text{LP}} \geq \sum_p \sigma_p + |S|\mu' = z_{\text{RMP}} + |S| \cdot \text{rc}_{\min}.$$

Substituting the pricing solve's own dual bound rc_{lb} , which never exceeds $\text{rc}_{\min}$, only weakens the inequality, and taking $\min(0, \cdot)$ leaves the bound at z_{RMP} once the master has priced out. Because the covering rows relax the partition and the partition model reproduces the visit count exactly, the chain $\text{LB} \leq z_{\text{LP}} \leq z_{\text{IP}} \leq$ the optimal visit count holds.

Stated in the form the main article reports, the bound is

$$\text{LB} = z_{\text{RMP}} + |S| \cdot \min(0, \text{rc}_{\text{lb}})$$

on the aggregated master and

$$\text{LB} = z_{\text{RMP}} + \sum_{s \in S} \min(0, \text{rc}_s^{\text{lb}})$$

on the station-indexed one, the same argument being applied to each convexity row in turn. Within the caps of the benchmark the bound is informative only at 50 SKUs, where the mean bound of 3,312 exceeds the trivial floor of one visit per order by 63% yet sits well below the mean of 7,590 visits found; from 500 SKUs upward it is vacuous, for the reason Section S8 records.

Three conditions make the argument valid and the implementation meets all three. Pricing is solved as an exact integer programme, so its dual bound holds even when a call is truncated.

The station-indexed repair is applied one convexity row at a time at a common dual vector, which the run enforces after the generation loop. The bound covers the multi-item supports, to which the singleton constant of Section 4.2.1 of the main article is added. Degeneracy of the covering master slows how quickly the floor rises as columns arrive but never invalidates it, and we declare convergence only when an exact solve at the true duals proves that no bundle of negative reduced cost remains.

S6 The Company A instance

Section 6 of the main article gives the decision pool and the workload basis in prose. This section records the dataset behind them, the residual resources the optimisation actually receives, and the two parameters the site fixes rather than we do.

Table S6: The Company A instance as it enters the solvers.

Quantity	Value
Planning horizon	13 weeks, 1 Sept. to 30 Nov. 2021
Orders	284,862
Order lines over the horizon	1,486,608
Products (SKUs)	21,874
free to relocate	15,975
frozen in place	5,899
Stations, of which evaluated	26, of which 24
Movable products held per station	8 to 2,575
Processing rate V_s , lines per unit time	6.2 to 10,400
Legacy station load $\sum_p L_p/V_s$	0.8 to 9,923
Workload budget T_s	110% of the legacy load

The two excluded stations hold 1 and 2 locations and carry 5 and 11 order lines between them. Processing rates span three orders of magnitude because the stations use different picking technologies, which is why workload is normalised by V_s and raw line counts are not comparable across stations.

Two parameters are the site’s operating rules rather than choices made here. A product is frozen when it appears in at most five orders *and* sits at one of the three static-shelving stations, the five-order threshold being the site’s own; and the workload budget T_s is 110% of a station’s complete legacy load, the 10% being the tolerance the site operates to. Neither is tuned here, which is why no sensitivity is reported over them.

Frozen products keep their slots and their workload share, so the optimisation runs on residual resources rather than full ones. Writing P_s^{fix} for the products frozen at station s ,

$$\bar{\zeta}_s = \zeta_s - |P_s^{\text{fix}}|, \quad \bar{T}_s = T_s - \sum_{p \in P_s^{\text{fix}}} L_p/V_s,$$

and the solver receives $\bar{\zeta}_s$ and $\bar{T}_s$ in place of ζ_s and T_s . Evaluation reverses this: every station is charged with its complete load, frozen and movable products together, over the complete stream of 284,862 orders rather than the 83,183 that contain a movable product. The reductions of the main article’s industrial table are therefore measured against the layout the site operates, on the whole catalogue, and are not inflated by the reduced decision pool.

S7 The industrial community-bound search

Section 6 of the main article selects the community bound β for Company A, whose stations hold between 8 and 2,575 movable products, on a seven-point grid rather than from the rule of Section 4.1. This section records the full search behind that choice.

Applying the rule $\beta = \max(5, \lfloor 0.4\zeta + \frac{1}{2} \rfloor)$ to the mean station capacity returns $\beta = 266$. That setting is feasible and removes 5.66% of the visits the site incurs today, so the rule does yield a deployable layout here. It is simply not the best value reachable, and nothing in the rule reveals as much.

Table S7 reports the fifteen settings evaluated. Feasibility is not an interval in β , and the shape it does take is the one the split rule predicts. A community larger than a station has to be split at placement, and the split rule then places products one at a time under the same admissibility test, so the blocks it produces are sized by what the stations can take rather than by β . Both ends of the range therefore respect the small stations: the low end because its communities already fit them, the high end because almost every community is split before placement. The middle is the worst place to sit, with communities too large for the small stations and too small to force the split path throughout. The four infeasible settings, from 33 to 133, are exactly that middle.

The protocol a site would run is the grid anchored on the mean capacity, $\{33, 67, 133, 200, 266, 333, 399\}$, keeping the feasible layout with fewest visits. Five of the seven are feasible and $\beta = 333$ is the best of them, which gives the heuristic row of the main article’s industrial table and is why that row’s time entry charges the whole grid, 790 seconds. The other eight settings were exploratory and are excluded from that cost; charging all fifteen would raise the heuristic to 1,480 seconds, still an order of magnitude below the next fastest method. One exploratory setting is better, $\beta = 300$ at 994,596 visits against 996,448, a further 0.19%, but a grid specified in advance does not contain it.

What the grid buys is not a finely tuned bound but the avoidance of the infeasible band. Above $\beta = 200$ every setting evaluated is feasible, and the eleven of them span 5.34% to 6.39%, a band of one percentage point: once the bound is large enough to force the split path the visit count is insensitive to it. Nothing therefore rests on the particular value the grid returns, and we do not read $\beta = 333$ as a calibrated quantity. The insensitivity is not total, because the settings differ more in how they leave the workload than in what they save: $\beta = 450$ reaches 6.31% at a utilisation dispersion of 21.6 against the selected setting’s 27.4. We keep the selection rule as stated rather than adding dispersion to it, since the rule was fixed before these runs and choosing on a criterion the outcome suggested would not be a protocol a site could follow in advance. On a warehouse of interchangeable stations none of this arises, the rule’s β being feasible on all 29 benchmark instances.

Table S7: The industrial community-bound search on the Company A instance.

β	Visits	Reduction (%)	Time (s)	Status
15	1,037,244	2.38	50	feasible
33	1,010,119	4.93	97	infeasible, grid
47	1,000,007	5.88	79	infeasible
67	996,734	6.19	105	feasible, grid
100	1,001,182	5.77	74	infeasible
133	1,007,309	5.20	123	infeasible, grid
200	1,001,428	5.75	89	feasible, grid
250	1,005,780	5.34	78	feasible
266	1,002,379	5.66	87	feasible, grid, rule
300	994,596	6.39	70	feasible
333	996,448	6.22	94	feasible, grid, selected
350	1,000,162	5.87	119	feasible
365	1,001,206	5.77	117	feasible
399	1,000,199	5.86	194	feasible, grid
450	995,426	6.31	104	feasible

Reduction is against the legacy layout’s 1,062,507 visits. A setting is feasible when no station exceeds its own workload budget, 110% of its complete legacy load; the check is per station, so a low visit count does not imply feasibility. “grid” marks the seven settings of the protocol a site would run, “rule” the value returned by $\beta = \max(5, \lfloor 0.4\zeta + \frac{1}{2} \rfloor)$ on the mean station capacity, and “selected” the feasible grid setting with fewest visits, which gives the heuristic row of the main article’s industrial table.

S8 Per-instance column-generation convergence record

The released archive named in the main article’s data availability statement carries, for every column-generation run, the convergence status, the restricted-master value z_{RMP} , the pricing dual bound rc_{lb} returned by the last pricing call, and the deadline the generation loop reached. Those records support the reading of Section 4.2.4 of the main article: no run proved that no improving bundle remained, the generation loop reached its $0.70B$ allocation on all 29 instances at all four sizes, and the last dual bound was strictly negative everywhere, from -696 to $-1,497$ at 50 SKUs and widening to -2.3×10^5 at 2,000 SKUs. The quantity that pushes the reported bound down is therefore the correction $|S| \cdot \text{rc}_{\text{lb}}$ rather than any weakness of the relaxation, and the binding constraint is the number of iterations reached inside the cap.